



04 作業系統

比爾蓋茲因為個人電腦作業系統的設計，而成為全球最富有的人，並且蟬聯多年（2003 年因為幣值更動使得比爾蓋茲拱手將寶座讓給 IKEA 家具的創辦人），這使人不禁好奇，作業系統到底是怎樣的一個法寶呢？

比爾蓋茲在 80 年代許下的企業願景為：每一個家庭都有一台個人電腦（Personal Computer；PC），每一台個人電腦上面都跑著微軟的 Windows 作業系統（目前的願景則為：未來，所有的企業、人們及裝置都可以互相聯結，而使用者不論任何時間、任何地點，使用任何裝置都可以使用網路服務）。環視今天的生活週遭，果然比爾蓋茲的預言成真，因此微軟也從 1975 年成立的小公司變成反托拉斯官司纏身的世界第一大軟體公司。微軟的興起及行銷策略也促使另一派人士推動自由軟體（free software）與開放原始碼（open source）。

本章節將首先介紹作業系統的基本觀念，介紹不同的系統及資源分配管理方式，最後在簡介幾個熱門的作業系統。

4-1

作業系統簡介

作業系統到底是甚麼呢？簡單地說，作業系統是一個程式，負責管理電腦裡的硬體及周邊設備，扮演介於使用者與電腦硬體的中間人。作業系統的存在目的是提供使用者一個方便又有效率的环境，使其能夠執行程式。我們知道任何一個電腦系統，大致上都可分為四部分：硬體（微處理器、記憶體及輸出入設備）、作業系統、應用軟體（我們常用的文書處理軟體及電動玩具等，或者是系統程式如組譯器、編譯器等）及使用者（人或其他電腦）（圖 4-1）。作業系統控制並且支配不同的應用程式使用各種資源，因此作業系統就像是電腦的管家婆，管家婆最重要的任務就是提供使用者方便。



圖 4-1 電腦系統的架構

不同家庭的管家婆，所注重的地方也不同。家有長者的家庭自是請管家婆多幫忙看顧老人；而在家有幼兒家庭中的管家婆，則是得負責照顧小朋友。作業系統也是如此，我們前面提到：作業系統要提供方便及效率。實際上，這兩點有相互矛盾的時候。這就像是你不可能叫管家婆要把房子徹底的打掃乾淨，卻又只能花很少的時間打掃。對於大型系統而言，為了提供諸多使用者同時使用，作業系統必須著眼於效能方面的考量，分配好使用資源；對於個人電腦而言，由於大部分的情況都是由電腦單一使用者掌控，因此，除了必須考量效能之外，資源分配就顯得比較不重要。但是，另一重點則在於容易使用（user friendly），功能的使用必須能讓使用者很快就上手操作，為了提供使用者親切的介面，只好犧牲一點效能來供應圖形化介面（Graphic User Interface；GUI）。在不同場景使用的電腦，作業系統就必須因時制宜，使用不同的作業系統來達到該場景的需求。

對於硬體而言，作業系統是直接接觸的程式，因此，這個管家婆得負責分配資源給有需要的應用程式及使用者。此外，作業程式也得負責防止因為使用者執行程式而導致的錯誤發生，或者是對電腦的不正確操作。整個作業系統是一個大程式，至於這個大程式裡應該有哪些功能，則沒有標準的定義。通常我們會認為，雖然不同的程式會使用到不同的資料及控制，但是其中有交集的部分，就應該由作業系統來負責，或者我們也可以說，作業系統是一個在電腦內部隨時都在執行的心核程式（kernel），而其他的則被歸類為應用程式。

如果更嚴謹一點，我們應該這麼說：整個作業系統是一群程式的集合，而作業系統中最重要的部分就是常駐在記憶體的核心程式，核心程式負責管理作業系統，也就是說，核心程式負責把其他部分的作業系統（非常駐程式）在必要時從硬碟中載入到記憶體裡（圖 4-2）。不論我們使用哪一種作業系統，當按下電腦電源時，核心程式就負責把其他系統載入到記憶體中，這個過程就稱為開機（boot）。

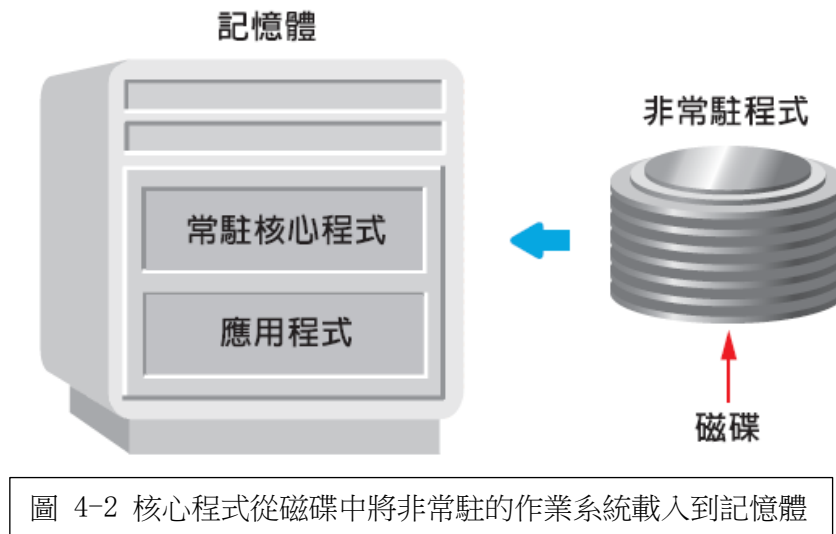


圖 4-2 核心程式從磁碟中將非常駐的作業系統載入到記憶體

中央處理器管理

中央處理器（CPU）是整個計算機的核心，所有的程式都會需要處理器做運算，當多個程式同時必須使用處理器時，作業系統要負責讓 CPU 充分發揮其功能，提高使用效率。當有一**程序**（process）處於等待的狀況時（譬如說是等待使用者輸入），作業系統就得讓 CPU 去做別的程序運算，而不能讓 CPU 也跟著等下去。此外，假設突然有一程序其優先順序較目前 CPU 在運算的程序高時，作業系統就得負責把 CPU 搶過來給優先順序較高的程序使用。

記憶體管理

雖然現在的主記憶體容量以 GB 計，但是不論如何還是有限，如何把這些有限的記憶體資源進行合理的分配，使每一個程序都滿足，並且提升效能，也是作業系統的重要任務。作業系統在分配記憶體時，要使各個程序與資料相互隔開，避免相互干擾。也就是說，要讓每個程序都能獨立執行，但是，也要能使每個程序共享公共的程序和資料，以避免重複的程式片段和資料佔用，節省記憶體空間。此外，當程序在執行時，如果發現記憶體空間不夠，作業系統要能做出適當的處理，以保證目前執行的程序能夠繼續進行。最重要的是：要防止使用者程序及程序執行時所需要的資料破壞作業系統本身，因為一旦作業系統遭到破壞，整個計算機系統就無效了。

檔案管理

由於主記憶體空間有限，因此大部分的程序和資料，及作業系統本身都是放在輔助記憶體上。如何標示這些資料檔案，如何有條理地組織這些訊息，使檔案能夠合理的存取與控制，使用者能夠方便而安全地使用，這些都算是作業系統得負責的檔案管理部分。

周邊設備管理

計算機一定有周邊設備讓使用者進行 I/O 輸入輸出，作業系統得負責有效率地管理各種周邊設備，最好還能**隨插即用** (plug-and-play)，並且要提供簡易的使用者介面程式，讓使用者就算在對設備不熟悉的情況之下，也能循著導引使用設備。

程序管理

每一個程序在作業系統之中都對應著一個**程序控制表** (Process Control Block ; PCB) (圖 4-3)，此控制表記載著該程序的相關資訊及程序的狀態。當程序進入系統之後，它們是被放在**工作佇列** (job queue) 之中，此佇列是由等待主記憶體配置的程序組成。位於主記憶體中，並且就緒。等待執行的程序則被存放在**就緒佇列** (ready queue)，作業系統負責源源不斷地將這些程序交給 CPU 進行運算。



圖 4-3 程序控制表 (PCB)

4-2 各類作業系統

隨著時間的演進，作業系統也不停地進步，使得軟體的發展能夠跟上硬體的進步速度，這樣才能充分應用不斷開創出的新硬體。我們接著要簡介各類型的作業系統以記作業系統的演進。

主機型系統

手動操作階段

早期的電腦體積非常地龐大，並且沒有軟體，也就是沒有系統來進行控制。一切得從控制太進行。輸入的裝置可能是讀卡機和磁帶機，輸出的裝置則為印表機、打孔機或磁帶機，這個階段的特色是使用獨占計算機的所有資源，並且直接使用這些硬體。使用者要執行程式時，首先要將計算機進行初始化 (就像是我們量體重的時候也要先把磅秤歸零)，然後啟動控制開關，將程式、資料和一些與工作有關的控制訊息打洞在打孔紙內，然後依序輸入進計算機中 (所以拿著打孔紙送進系統時一定要非常小心，若不慎跌倒，打孔紙全散，可就要多費功夫了)。計算機接收到輸入之後，

就開使執行作業。一段時間後（可能是數分鐘、數小時、甚至是數天）就可以看到輸出。如果程式執行的過程中有錯誤，還必須要檢測記憶體和暫存器裡的值以除錯。

我們可以想見當時要使用計算機是多麼的不方便，從輸入作業到作業完成的整段期間，每一個操作步驟都需要使用者直接控制，並且一個作業還沒完成之前，是不能穿插別的作業進來一起計算。由於手動操作的速度是緩慢的，再加上使用者獨占計算機，因此計算機大不分的時間都處於等待的狀況，造成資源浪費，無法發揮計算機的功能。此外，由於使用者要直接接觸到硬體，因此使用者得熟悉計算機的各部分細節及操作方法，上手難度高，而且操作麻煩又容易出錯。

批次系統

後來我們有了**批次系統**（batch system），這個早期的作業系統設計很簡單，它主要是負責一個工作到下一個工作時自動地控制轉換。批次系統由於有了作業之間的轉接自動化，因而縮短了作業之間等待 CPU 的時間，終於較能發揮計算機的處理能力。

批次系統的情況中，作業系統是常駐在記憶體中（圖 4-4），作業員把若干個有相同或類似的工作集合為一整批（batched），然後將整批作業讀入主記憶體的緩衝區，再轉到磁帶上。這工作完成之後，就由系統監控程式接手，負責將磁帶上的每個作業循序讀入主記憶體中，交給 CPU 去運算處理。一個作業接著一個作業運作，直到整批全部完成，再把下批作業讀入磁帶，以同樣的方式進行處理。因此，我們可以看到，處理一批作業的時候，由於各個作業之間的轉接是由監控程式自動操作，可以縮短手動操作緩慢所造成的 CPU 等待時間。



圖 4-4 簡單的批次系統記憶體配置

但是這一類型的計算機中，CPU 仍然經常處於閒置等待的狀態。這是由於 I/O 緩慢的關係。因為機械式的輸出入裝置處理資料速度本來就比電子裝置慢，即使是一個慢速的 CPU 每秒也能處理百萬個指令，但是，最快速的讀卡機每分鐘也僅能讀 1200 張卡而已。雖然讀卡機的技術也在進步，可以加快讀卡的速度，但是同時 CPU 的運算速度也在成長，導致讀卡機反而越來越跟不上 CPU。

不過不管如何，批次系統已經允許把所有工作都一起放在磁碟中，不必串列在讀卡機上，如果能夠直接存取幾件工作，作業系統就可以進行工作排班（job scheduling），進而增進效率。此外。批次系統也讓使用者不必再理解計算機的每一個細節就能進行處理。

多元程式規劃系統

有了工作排班之後，最重要的就是多元程式規劃（multiprogramming）的能力。由於一個程序不太可能讓 CPU 一直處於忙碌的狀態，因此多元程式規劃的目的就是要增進 CPU 的使用率，只要還有程序需要 CPU，CPU 就不會閒置等待。

一個程式被載入記憶體中並且執行時就稱為程序，一個程序的生命週期有幾個狀態以表示程序目前的動作。各個狀態之間的關係（圖 4-5），一個程序可能有以下幾種狀態：

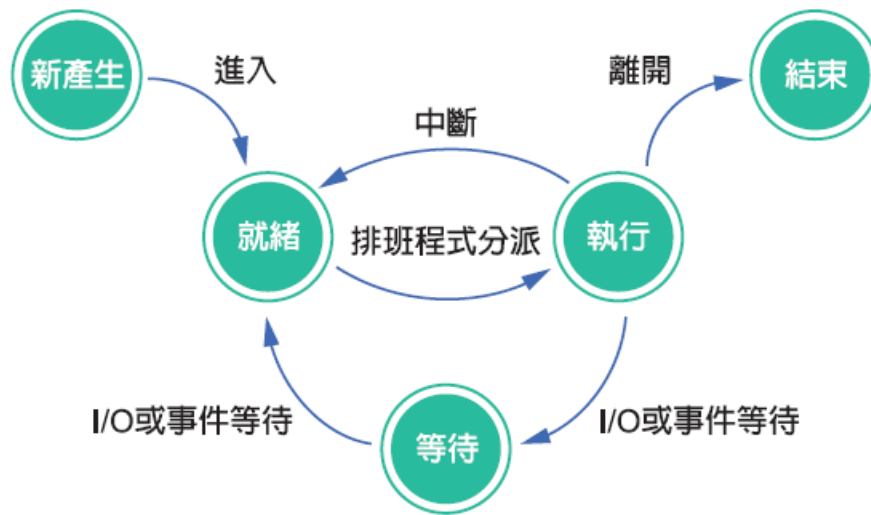


圖 4-5 程序狀態關係圖

- **新產生**：程序正在產生中。
- **執行**：程序得到資源正在執行中。
- **等待**：程序在等待某個事件發生，譬如說是等待是用者輸入。
- **就緒**：程序一切已準備就緒，在等待處理器執行。
- **結束**：程序已完成。

在多元程式規劃系統中，只要程序進入等待的狀況時，CPU 就轉移到其他工作上運作，如果原本在等待的程序已經變成就緒，就有機會再重新得到 CPU 資源。這樣的觀念就像是銀行櫃檯小姐，每次只能服務一位客戶，可是如果遇到客人需要填單時，為了提高服務效能，就請客戶先在旁邊填單子，然後櫃檯小姐先行處理下一位客戶的資料。

所有的工作進入系統後，都放在**工作池** (job pool) 中，多元程式規劃必須為使用者決定執行的優先順序，將多個程序放置在記憶體中 (圖 4-6)。多元程式規劃中有兩個主要的排班，一是**工作排班** (job scheduling)，另一個是**處理器排班** (CPU scheduling) (圖 4-7)。當有多個程序需要執行時，會先放到大型儲存裝置 (一般來說是磁碟) 的工作池，在工作池中等候被執行，工作班負責把程序從工作池中選出，載入到記憶體內以便執行。可是程序進入記憶體後也是得等待，這時候就要使用 CPU 排班從記憶體中挑選已經準備就緒的程序，好將 CPU 分配給它使用。



圖 4-6 多元程式規劃系統的記憶體配置

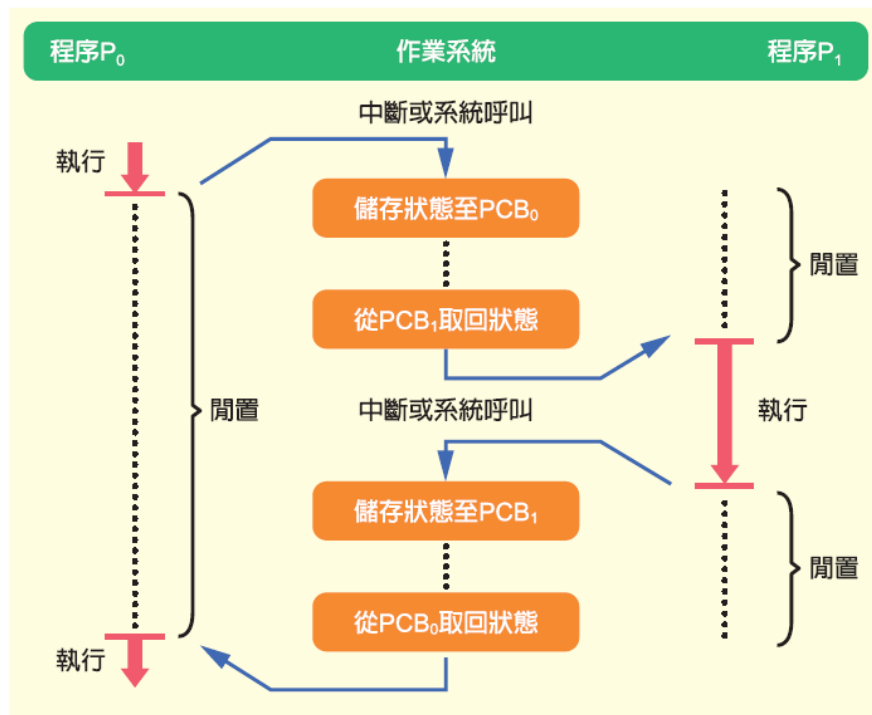


圖 4-7 CPU在程序之間來回運轉狀況。CPU首先為P0進行運算，P0進入等待狀況之時，作業系統提出中斷或者系統呼叫，接著CPU改為P1進行運算，如此來回轉換。

這兩種排班除了執行式的差別之外，最主要的差別在於執行的次數。CPU 排班必須經常為 CPU 選擇新的程序。因為一個程序可能已毫秒之後就因為有 I/O 要求而進入等待的狀況。所以 CPU 排班執行的頻率非常頻繁，並且必須在很短的時間內就作出決定，否則將會發生大部分的時間都浪費在作決定上面。排班有一些不同的決定方式使用，我們將在後面介紹一些。

分時系統

雖然多元程式規劃系統可以提供不同系統資源的有效利用方式，但是對程序來說，一次還是只有一個程序能被執行。分時系統則可以讓大家都認為自己與電腦正在交談沒有間斷。分時系統的方式是將 CPU 的時間切割成很多小時段，CPU 不停的在許多程式或者使用者之間切換來執行許多工作。因為切換的很頻繁，所以每個使用者都以為自己和 CPU 是持續運作的（圖 4-8）。

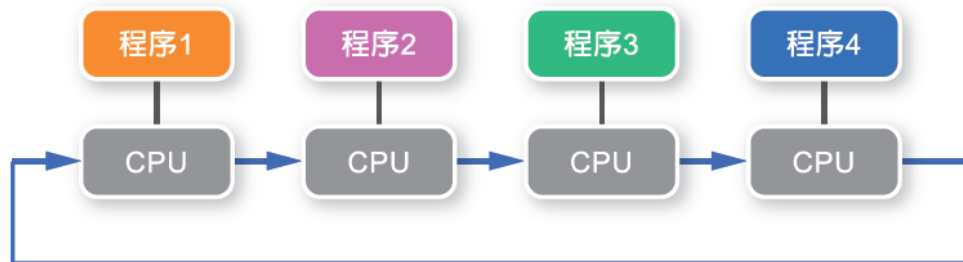


圖 4-8 時間一到，CPU就為下一個程序進行運算，如此不停來回處理多個程序

由於數位處理器的運算速度非常快，而我們人類在輸入或處理資料時，對電腦而言，卻相對地太慢了，所以有分時的概念產生。同樣的場景也可能發生在天才與一般人之間。有一年，旅日圍棋高手林海峰棋士曾同時和約幾十位棋士下快棋，林海峰因為頭腦轉得快，每棋盤只看十秒鐘就可決定下一個棋步，他就這樣來來回回和各個棋士對下，每個棋士大約可以想數分鐘再決定棋步，對他們而言，林海峰好像只和他下棋，可是林海峰實際上卻是同時和很多人對下啊！這就是一種分時的概念。

像是我們平常連上工作站，就是一個分時系統，同一時間點可能有很多使用者同時登入，跑程式、進行文字編輯、或者收發信件、或是 telnet 到 bbs 上去，也可能是 ftp 到某個站台，一群使用者能夠同時在機器上進行不同的工作，就是拜分時系統之賜。分時系統有幾大特色：

- **同時性**：可以同時有若干個使用者連結到同一台計算機上進行運算。
- **獨立性**：不同的使用者之間並不會相互干擾。
- **即時性**：每一個使用者都可以即時得到計算機的回應。

分時作業的概念，不僅可讓多個使用者共用一部電腦；也讓我們能在只用一個處理器的個人電腦上，同時開多個視窗來完成不同的任務。而為了要在同一段時間內完成使用者交辦的多項任務，有效且公平的排程也是作業系統設計的重點，唯有好的排程，才能繞電腦日理萬機而不脫線。

總結說來，多元程式規劃系統是倚靠**事件觸發** (event-driven)，也就是基本上程序都能不斷地執行，直到進入等待的狀況之下，CPU 就被分配到別的程序去使用；而分時系統則是倚靠**時間觸發** (time-driven)，不管程序在什麼狀態，時間一到 CPU 就進行下一個程序的運算，而原本的程序就必須等待，只不過因為 CPU 切換得很快，所以就好像 CPU 不曾離開過。

個人電腦系統

我們前面曾經提過，個人電腦系統的設計方針與主機型系統並不同。個人系統的目標在於增進使用者操作方便，並且提昇 CPU 的回應速度避免讓使用者等待。

個人電腦上最常見的作業系統有早期的 DOS (磁碟作業系統)，這是微軟最早的作業系統產品，用在 IBM 相容個人電腦上，讓使用者以指令的方式來操控電腦。第一個圖形化介面的個人電腦作業系統是 Mas OS，適用於蘋果電腦上。隨後微軟也推出 Windows 1.0，成為 PC 上的第一個圖形化作業系統。

另外，還有根據大型電腦作業系統 UNIX 所發展出的 Linux 也是個人電腦上受歡迎的作業系統，早期就像 DOS 一樣只有命令列模式，現在也有了親切的圖形化使用者介面。我們將在本章節最後簡介這些個人電腦上的作業系統。

多處理器系統

大多數的系統只有單一處理器，可是有些系統有一個以上的處理器，彼此之間緊密的溝通合作、共享資源、共用時脈，這類型的系統就是**多處理器系統** (multiprocessor system) (圖 4-9)。

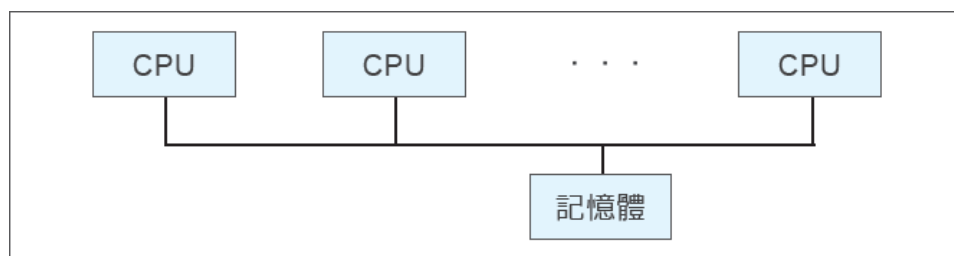


圖 4-9 多處理器系統中，多個應用程式共用記憶體等資源

使用多處理器系統也幾個好處，最顯著的好處是因為我們增加了處理器，所以當我們在進行大量運算時，各個處理器可以一起分擔工作。然而，值得注意的是，假設我們添加了四顆 CPU，變成原來的五倍，但是我們不能天真的以為效能就會變成原來的五倍。因為多顆處理器一同運作時，一定要多費一些功夫在保持工作進行無誤，處理器之間溝通合作正常。這跟生產線運作的情況不同，生產線如果多開一條，產量就增加一倍。雖然沒有辦法呈現等倍數成長，不過多處理器還是能增進效能。

想要增進效能，其實也可以採用生產線的模式，用幾台單一處理器系統。然而從經濟層面來考量，我們就會發現多處理器系統較符合經濟利益。因為多個處理器可以共用周邊設備、共享電腦資源，並且空間上也較不佔位子。

此外，因為有多個處理器一起運作，可以達到容錯 (fault tolerance) 的目的。所謂容錯，是指如果有錯誤發生，也不會礙了大事。生活上我們也常有這樣的準備。假設我們要去參加一個重要會議報告，那我們就會把要報告的檔案拷貝分放在硬碟、隨身碟、甚至網路磁碟也在放一份。如果臨時電腦當掉無法開機，那我們借用別人的電腦讀隨身碟裡的檔案，萬一隨身碟也不小心粉碎了，那至少網路上還放了一備份，最後我們還是能成功地完成報告而不受這些錯誤影響，這就是容錯。在多處理器系統下的容錯是指如果各個工作能夠適當地被分配在不同的處理器上，那如果有其中一個處理器發生問題時，系統也不會因此而中止，只是會使得速度變得較原先來得慢。

分散式系統

簡單來說，分散式系統指的是電腦是分散的，也就是說，電腦們並不共享資源或者時脈，每個電腦都有自己的記憶體甚至是各自的作業系統，彼此之間依靠網路傳輸交換資料。雖然電腦會透過網路連線使用其他電腦的資料或元件，但是電腦前的使用者不會有任何感覺，對使用者而言，就好像只是跟單一電腦交談。

由於我們已經進入網際網路的時代，資料的傳輸多半使用網路，再加上擁有高速網路環境，所以分散式作業系統的使用環境已經越來越成熟。其中 Web Service 則是將傳統分散式系統演變成網際網路上的標準化服務，它透過網際網路通訊協定及資料格式的開放式標準 (例如：HTTP、XML 及 SOAP 等) 來為其他的應用程式提供服務，達到分散式的效果 (圖 4-10)。

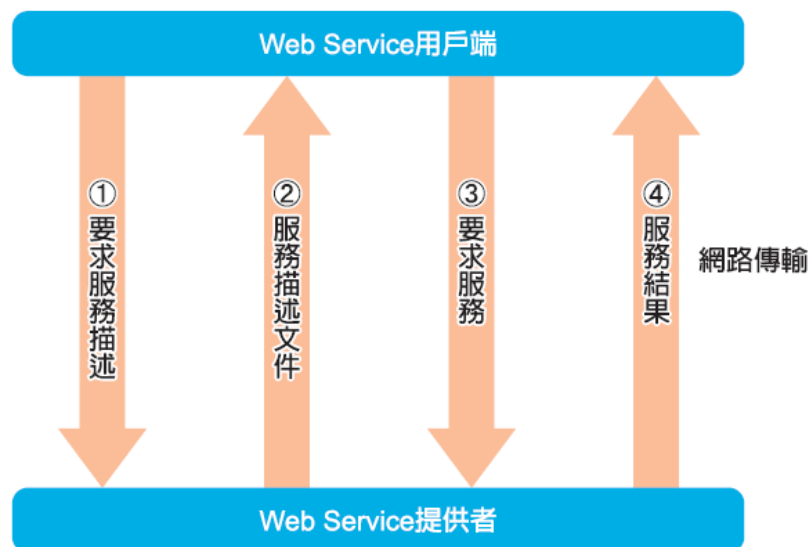


圖 4-10 分散式應用的 Web Service，透過網路提供服務

Web Service 能夠提供給企業許多便利性，假設要成立一個旅遊網站，早期我們可能要理解飛機訂票系統、鐵路訂票系統、以及飯店訂房系統。如果要一個個去了解如何建置這些系統可能會大傷成本，這個時候如果採用航空公司等提供的訂票訂房 Web Service，則我們就不需要費心思理解內部怎麼運作，只需要設計好自己的介面。程式運作的時候，就會透過網路連線使用分散在網路其他位址的服務元件。由於分散式系統資料在網路上傳來傳去，因此資料的安全性將是一大考量。

即時系統

即時系統為一特殊目的作業系統。在即時系統中，計算機要能即時回應外部事件的要求，並且於規定的時間內完成對該事件的處理，還要控制所有的即時設備和即時工作能夠協調一致地執行。也就是說，即時系統的使用情況為使用者對於處理器操作或資料的傳輸在時間上有嚴謹的要求，通常是在專門應用範圍的控制裝置，感應器將資料傳送給電腦，電腦必須將資料加以分析，並且控制感應器。

即時系統有嚴謹的時間限制，這也是它的主要訴求，作業一定得在所定義的時間內完成，不然將會當機。使用即時系統可能是用來控制科學實驗、醫學影像系統、或者是工業工程控制。這些情況下都必須有嚴謹的時間控制，譬如造飛機的時候，生產線上每一個環節的時間點都必須剛剛好，總不能機頭都已經往前推進了，噴槍才開始噴漆，或者更嚴重的情況可能是機器手臂動作太慢，無法跟其他工作協調導致傷害到飛機，這些情況都是不允許發生的。

因此我們可以知道，在設計即時系統時，有幾個重點：

- **即時時鐘管理**：用以完成定時工作或者延遲工作，以利與其他工作協調。
- **過載保護**：當工作太多導致計算機來不及處理的時候，必須要有完善的服務策略，例如使用緩衝區來應急，將優先順序較低的工作暫時緩一緩先存起來，使優先順序較高的工作能先執行。
- **高度可靠性**：必須保障計算機發生問題時，系統仍然正常工作，通常使用的方式就是如上述之多處理器方式解決。

4-3

CPU 排班

我們在上一節曾經提到多元程式規劃作業系統，而其基礎便是 CPU 排班。藉由 CPU 在不同的程序中轉換，各個程序就可以盡可能地榨乾 CPU 的運算能力。也就是說，多元程式規劃作業系統的主要目的，就是藉著 CPU 排班以保持隨時都有一個程序在執行，提高 CPU 的使用率。在單一處理器系統中，一次只能有一個程序被執行，其他的都得依照 CPU 排班在旁等待。

CPU 排班的決策時間點可能發生在下面五種情況（圖 4-11）：

1. 程序新產生時
2. 程序從執行狀態變成等待狀態（譬如有 I/O 要求）
3. 程序從執行狀態變成就緒狀態（譬如有中斷發生時）
4. 程序從執行狀態變成就緒狀態（譬如 I/O 要求得到回應）
5. 程序中止時

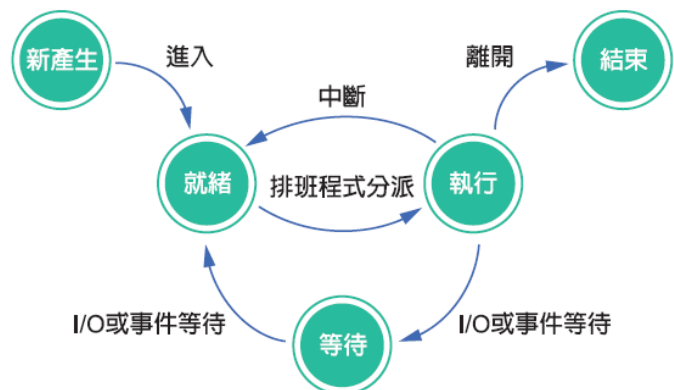


圖 4-11 五個必須 CPU 排班的时间點

對於情況 1 與情況 4 而言，為了讓 CPU 使用率增高，所以勢必得進行 CPU 排班，挑出一個新的程序給 CPU 運算。但是情況 2 跟情況 3 則不然。有些排班方式確保已經享有 CPU 資源的程序能夠一直用，不管其他程序的狀態，直到程序自己跳到其他非執行的狀態，才進行排班，這種稱為**不可搶先排班** (nonpreemptive)；另一類型則為**可搶先排班** (preemptive)，也就是時時刻刻都要注意所有程序的狀態，如果有程序進入就緒的狀態，那就要把已經就緒的程序跟正在使用 CPU 的程序相互比較優先順序。優先順序較高的就能把對方踢去排隊。

接下來我們要實際介紹各種 CPU 排班演算法，可是，在那之前，我們要先介紹幾個用來評估各個排班演算法好壞的標準：

- **CPU 使用率**：由於我們希望 CPU 越忙越好，因此理論上 CPU 使用率最好達100%，然而實際應用上，CPU不可能到達此極限。
- **產能**：如果 CPU 越忙碌，**產能** (throughput) 就會越高。評估產能的方式可以是算計單位時間完成的程序數目，然而這樣並不盡公平，因為如果執行到大程序，可能只能完成幾個；相對的，如果執行的都是小程序，則產能可能就大得多。
- **回復時間**：**回復時間** (turnaround time) 也就是從頭到尾所花的時間。產能是衡量單位時間 CPU 可完成多少程序，而回復時間則是指一個程序從進入電腦開始算起，經歷了等待主記憶體、在就緒佇列中等待、I/O 處理時間、及 CPU 執行，一共花費多少單位時間。
- **等待時間**：算計程序會花多久的時間在就緒佇列中等待。這個標準比回復時間更客觀，因為假設有一個優先順序很高並且又很龐大的程序，可能回復時間就會很長，然而實際上回復時間中，大部分都是在享用 CPU 資源。

接下來我們就要來看不同的 CPU 排班演算法，用來決定哪一個程序可以享用 CPU 資源。

先到先處理

最簡單的方式就是**先到先處理** (First Come First Serve；FCFS)，也就是把 CPU 分給第一個需要 CPU 資源的程序，我們可以用**先進先出** (first in first out) 特性的佇列來排置程序。每當有一個新的程序產生並需要 CPU 資源時，它就會被串接到就緒佇列的末端。當 CPU 完成一個程序，就把就緒佇列的第一個程序拿出來運算，並且把它從就緒佇列中踢除。這種排列方式是不可搶先的，也就是一旦 CPU 分配給某個程序，該成就一直佔用 CPU，直到它進入等待的狀態或是結束，CPU 才會執行下一個程序。

舉一個例子來看，假設程序的資訊 (表 4-1) 所示，時間以毫秒為單位：

表 4-1 先到先處理程序的資訊

程序	抵達順序	所需時間(毫秒)
P1	1	15
P2	2	3
P3	3	3

使用先到先處理的方式，整個 CPU 的工作表就 (圖 4-12) **甘特圖** (gantt chart) 所示：



圖 4-12 甘特圖

分析一下這種排班方式，我們會發現各程序的等待時間（表 4-2）所示：

表 4-2 各程序的等待時間

程序	等待時間(毫秒)	平均等待時間(毫秒)
P1	0	
P2	15	
P3	18	11

同樣者個程序，假設來的順序不同，就有截然不同的結果，（表 4-3）所示：

表 4-3 假設三個程序來的順序不同

程序	抵達順序	所需時間(毫秒)
P1	3	15
P2	1	3
P3	2	3

這種情況下的甘特圖（圖 4-13）所示：



圖 4-13 假設三個程序來的順序不同的甘特圖

各程序的等待時間則為，（表 4-4）所示：

表 4-4 假設三個程序來的順序不同的等待時間

程序	等待時間(毫秒)	平均等待時間(毫秒)
P1	6	
P2	0	
P3	3	3

同樣是這三個順序，帶示先來後到的順序不同，平均等待時間就相差很大。因此我們可以知道，雖然先到先處理方式在實作上很容易完成，可是卻不是最理想的排班方式。

最短工作先處理

從先到先處理的排班方式中，我們看到了如果大型的工作排在前面，會使得小型工作卡在後面拉長平均等候時間，因此最短工作先處理排班方式就能夠解決這個問題。所謂**最短工作先處理**（Shortest Job First；SJF）是說，當 CPU 排班為 CPU 挑選下一個執执行程序時，會檢視所有需要 CPU 資源的程序各會花多少時間完成其工作。最短的程序就能夠先得到 CPU 資源。其餘的等到下一次挑選時再進行比較。如果有兩個以上的程序所需時間相同，則再採用先到先服務的方式挑選。

舉個例子來說明，(表 4-5)：

表 4-5 最短工作先處理程序的資訊

程序	等待時間(毫秒)
P1	7
P2	3
P3	5

這個例子之下，甘特圖將如下表示，(圖 4-14)：

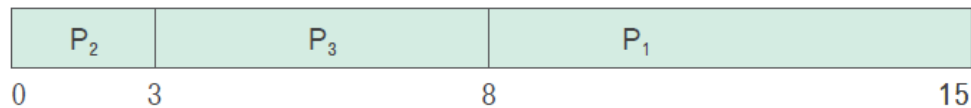


圖 4-14 甘特圖

各個程序的等待時間則為(表 4-6)：

表 4-6 各個程序的等待時間

程序	等待時間(毫秒)	平均等待時間(毫秒)
P1	8	
P2	0	
P3	3	3.67

如果使用先到先處理的方式，則程序先後順序及其平均等待時間(表 4-7)所示：

表 4-6 各個程序的等待時間

程序先後順序	平均等待時間
P1 → P2 → P3	5.67
P1 → P3 → P2	6.33
P2 → P1 → P3	4.33
P2 → P3 → P1	3.67
P3 → P1 → P2	5.67
P3 → P2 → P1	4.33

我們可以發現，除了 P2 → P3 → P1 剛好與最短工作先處理的順序相同之外，其餘的順序平均等待時間都會比 SJF 來得長些。最短工作先處理方式又可分為可搶先及不可搶先。如果是可搶先的情況，在 CPU 執行某一工作的同時，如果有新的程序產生，就要比較新產生程序的所需時間，如果較目前正在執行的程序之剩餘時間還短，新的程序就可以搶走 CPU，如果是在不可搶先的情況之下，則新的程序必須等目前正在執行的程序完成。

如果單純考慮使平均等待時間降到最低，那麼最短工作先處理的方式會是最理想的。雖然將一個短程序排到前面，會是使長程序擠到後面而增加其等待時間，然而卻可以大大減低短程序的等待時間，因此平均起來還是划算的。然而，最短工作先執行的難處在於實際執行時，我們很難能夠精準預測每一個程序將花多少時間來完成工作。

優先權排班

所謂**優先權排班** (Priority Scheduling)，是指給程序們階級化，標上優先順序。先到先處理的方式並沒有優先權的概念，但是最短工作先處理的方式則算是優先權的特例，也就是給予需時較短的程序較高的優先權先行使用 CPU。

假設有 (表 4-7) 的程序及其優先權：

表 4-7 程序及其優先權

程序	優先權	所需時間(毫秒)
P1	3	7
P2	1	5
P3	2	4

使用優先權為排班依據，則甘特圖 (圖 4-15)：

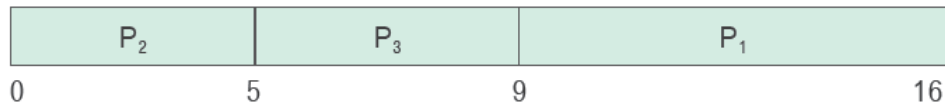


圖 4-15 優先權為排班依據之甘特圖

優先權可以是從多面向來算計，譬如可以計算其所需時間、記憶體需求、程序大小等來考量，也可以是根據程序的重要性，譬如說是緊急需要計算解果的程序，就可以有較高的優先權。優先權排班也可分成可搶先及不可搶先兩種，同樣地，如果有一個新的程序產生，並且其優先順序較目前執行的程序高時，在可搶先的情況下，就可以搶用 CPU；而在不可搶先的情況，就得等目前的程序處理完畢。

使用優先權來作為排班考量，最擔心的事情是程序會因為始終得不到 CPU 資源而變成**飢餓** (starvation) 的狀況。工作繁重的電腦系統中，如果使用優先權排班，很有可能會有一連串高優先權的程序源源不停地進入就緒佇列，導致低優先權的程序永遠停留就緒佇列中癡癡等待。解決這種困擾的方式就是把等待時間也列入優先權設定的考量中，也就是說，我們可以設定等待超過一定的時間，就提升該程序的優先權，如此一來，終有等到 CPU 的一天，不會發生年華老去卻始終不曾招致青睞的飢餓情況。

依序循環排班

前面提到的三種排班方式，雖各有好處，卻都不適合分時系統。分時系統的宗旨是要時間一到就得讓下一個程序使用 CPU。因此，為了分時系統而設計出依序循環排班的方式。這方式基本精神與先到先處理相同，然而不同的是在時間上做了把關，好讓各程序交還使用 CPU。

依序循環排班 (Round Robin Scheduling) 方式在使用時，先預設好經過多少時間 CPU 就該切換執行下一個程序，也就是設定好**間隔時間** (time slice)。所有的程序放在先進先出的佇列裡面，首先 CPU 排班從佇列裡挑第一個程序執行，然後開始進行倒數，時間到的時候就得讓 CPU 處理佇列裡下一個程序。然而我們得特別注意一下，依序循環排班方式可能有兩種情況會發生：

- **程序所需時間小於間隔時間**：在這種情況下，程序一執行完畢後，就得自動交出 CPU 使其執行下一個程序。
- **程序所需時間大於間隔時間**：程序執行時間比間隔時間大，在倒數完畢時，必定會有尚未完成的部分，但是我們必須堅守時間到換程序的遊戲規則，所以作業系統得把目前執行的內容及狀態記錄下來，寫到前面所提到的程序控制表 (PCB) 中，然後把該程序放到佇列的尾端，然後從佇列開頭拿取下一個程序執行。

舉個例子來看，假設我們設定時間間隔為 5 毫秒，並且有 (表 4-8) 的程序：

表 4-8 時間間隔為5毫秒

程序	所需時間(毫秒)	抵達順序
P1	17	1
P2	3	2
P3	8	3

則使用依序循環排班方式之甘特圖 (圖 4-16)：

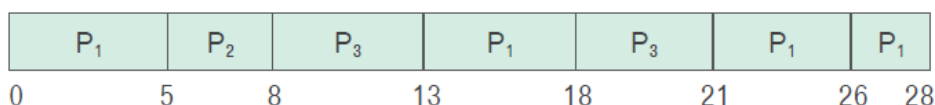


圖 4-15 依序循環排班方式之甘特圖

上圖中，由於 P2 並不需要 5 毫秒那麼多，3 毫秒執行完後，就讓 CPU 執行下一個程序，然後到了 18 毫秒，由於 P2 已經完成，不再放在佇列中，因此佇列中的頭將是 P3，故執行 P3 並將 P1 放進佇列中的尾端 (由於已經沒有其他程序了，所以同時也是佇列的頭)，最後到了 21 毫秒，CPU 就執行到了 P1，在 26 毫秒時，由於其他程序都已完成，佇列以空，因此就繼續執行 P1 直到完成。使用依序循環排班的方式，由於每個程序時間一到 CPU 資源就會被搶走，不能繼續霸占下去，因此依序循環排班的方式算是可搶先的排班。

依序循環排班方式中的關鍵在於時間間隔的選擇，下面 (圖 4-16) 我們顯示時間間隔對於程序的影響：

程序所需時間：8毫秒	時間間隔(毫秒)	執行次數
	10	1
	5	2
	3	3

圖 4-16 時間間隔對於程序的影響

執行的次數越多，代表程序轉進轉出 CPU 的次數越多，就會花較多的時間在讀取及寫入程序控制表，造成額外的負擔。如果時間間隔設定的太小，就會導致執行次數太多，相反的，如果時間間隔設定太大，就會變成跟先進先出相同的排班方式，因此我們通常使用「二八原則」，也就是讓 20% 的程序時間大於所需間隔，而 80% 的程序所需時間會小於時間間隔。

不論是採用哪一種排班方式，當程序需要資源時，都會循著**請求** (Request) 資源，**使用** (Use) 資源、**釋放** (Release) 資源的程序。如果在「請求資源」時，無法及時得到所請求的資源配置，則程序便進到「等待」狀態，直到順利得到資源，即使開始「使用資源」，該程序便可執行掌控所配置的資源。執行完畢後，必須「釋放資源」，好讓資源能被其他程序使用。

在**多元程式規劃** (multiprogramming) 中，程序之間彼此需要相互競爭已取得運算系統中有限的資源，搶不到資源的程序，就進到「等待」狀態，直到所需要的資源被釋放出來。但是程序進到「等待」狀態之後，也可能從此就陷入「等待」的狀態無法逃脫，因為它所請求的資源，被其他同樣在「等待」狀態中的程序所佔用，這就稱為**死結** (Deadlock)。在日常生或中也可能發生死結的情況，(圖 4-17) 所示，在十字路口中，因為大家都在等待其他方向的車輛空出車道，結果導致整個卡死。在死結中的程序，由於每個程序都在等待其他程序觸發事件，但是卻沒有一個程序可以先觸發事件，導致所有程序癱瘓，最終呈現餓死 (starvation) 的狀態。



圖 4-17 程序所請求的資源被其他同樣在「等待」狀態中的程序所佔住，即可能發生死結，就像是十字路口的車輛都在等待別的车辆空出車道。

4-4

記憶體管理

資源管理除了把 CPU 分配給各個程式之外，另一大重點在於管理記憶體。記憶體管理主要是讓記憶體能夠充分被利用。我們在前面理解到，為了使 CPU 的使用效率增加，我們必須把多個程序放在佇列裡等待，也就是說，我們必須把許多程序放在記憶體中，因此各個程序必須共用記憶體。記憶體的管理必須做到如以下之功能：

- **記憶體管理**：由於每一個程序都需要有一定的記憶體空間存放程序本身或者是資料，因此，記憶體管理一定要把記憶體**分割** (partition) 成各個區塊，以分配給各個程序，甚至是各個使用者使用。
- **記憶體位址定位**：使用者在執行程序時，完全不知道程序會被放在哪裡執行 (當然也不見得會關心)，程式撰寫員在執行自己所開發出來的程式時，也不知道自己的程式會放在記憶體的甚麼位置，甚至每次執行的時候，所放置的位址也可能都不同 (因為每次記憶體的使用情況可能都不同)，作業系統的記憶體管理功能必須要能負責把程式所使用的**邏輯位址** (logical address) 與記憶體裡的**實際位址** (physical address) 做**映射** (mapping) 的工作 (圖 4-18)。

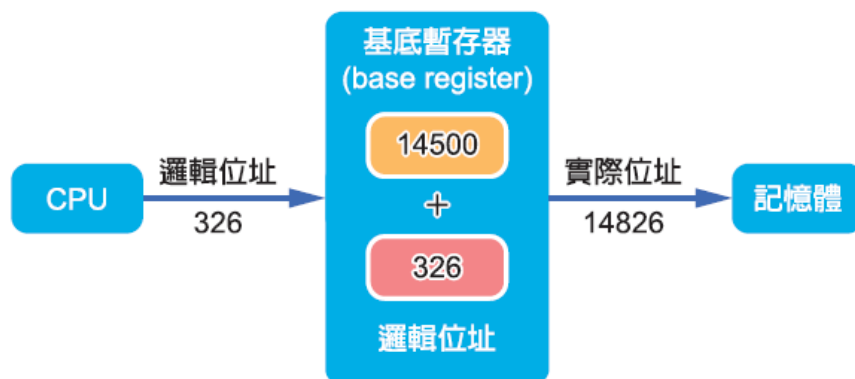


圖 4-18 邏輯位址轉為實體位址：透過基底暫存器進行轉換

- **記憶體保護與共享**：由於各個程序和作業系統都存放在記憶體中，因此，程序之間所使用的記憶體不能夠相互干擾。另一方面，記憶體中放作業系統的部分卻要讓各程序共用。此外，特別要小心的是要保護好作業系統所使用的記憶體區塊不會被程序破壞。

單一連續記憶體配置方式

最早期的記憶體管理方式是不支援多元程式規劃的，在這種方式中，概念上記憶體被分為三個連續的區塊，最底層是作業系統存放的區塊，其他部分則為隨著工作負荷被動態的應用程式佔用的區塊及未使用的區塊（圖 4-19）。

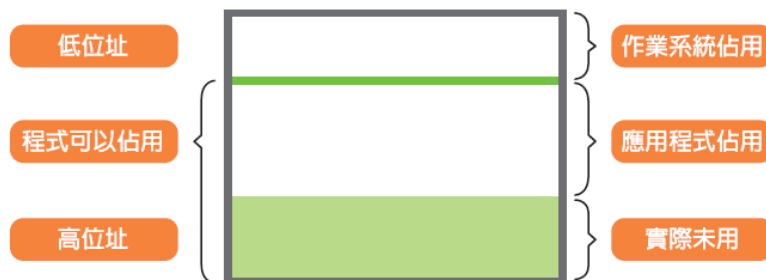


圖 4-19 最早期的單一連續記憶體配置方式

在這種記憶體管理方案中，為了防止使用者破壞（不論是有意還無意）作業系統，需要一些基本的保護機制。以下我們要介紹的是界線位址保護法。界線位址保護，是靠界線暫存器及基底暫存器來保護。如果作業系統是被放置在高位址區域，那界線暫存器就用來保護作業系統區塊最下界。當 CPU 在存取應用程式區塊時，首先會檢查存取的邏輯位址是否小於界線暫存器。如果發現所存取的位址比界線暫存器內的值還高，則發出錯誤中段訊息，要求作業系統做出錯誤處理，如果沒問題的話，則再加上基底暫存器的值形成時實際位址（圖 4-20）。如果是作業系統在使用記憶體，則不受界線暫存器的限制，甚至可以改變界線暫存器內的值。

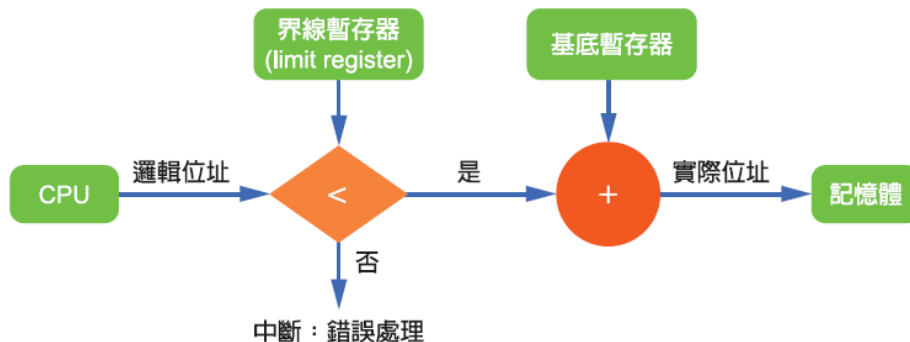


圖 4-20 利用界線暫存器和基底暫存器來提供記憶體保護

動態載入

前面談到的單一連續記憶體配置，程式和資料都必須在記憶體中存放以執行，如此一來，程序大小就會受限於記憶體的大小。動態載入則可以提供較大的彈性。動態載入是指常式 (routine) 只有在被呼叫的時候才會載入，也就是說，將所有的常式以可重定位載入的格式存放在磁碟空間內。主程式存放在記憶體中，執行的時候，如果需要呼叫其他常式時，首先查看是否已經存在記憶體內，如果不是的話，便呼叫重疊定位連結程式，將需要的程式或資料載入記憶體中，好讓程式能夠執行。

覆蓋

動態配置可以讓我們更彈性的使用記憶體空間，覆蓋則是另一種能夠讓大小超過記憶體容量的程式執行的方法。所謂覆蓋是指最主要的部份會一直放在記憶體中，可是那些只有在特定時後才需要用到的指令或資料，就只有被用到的時候才會放在記憶體中，其他時候就被覆蓋掉，藉著舊的指令被覆蓋，新的指令才有空間存放。

舉個例子來看：假設一編譯器在處理程式時，第 1 次處理的時候要建立符號表，然後第 2 次處理的時候才能產生組合語言。假設第 1 次處理的程式碼只有 80KB，第 2 次處理的程式碼有 70KB，所建立的符號表有 20KB，會被呼叫到的常式有 30KB，一共有 200 KB。假設記憶體只有 150KB，如果我們用動態配置也不夠使用，因此採用覆蓋的技術。仔細分析一下上面的片段，除了符號表和常式會一直用到之外，第 1 次處理的程式碼跟第 2 次處理的程式碼分別獨立使用一次，因此我們的策略是第 1 次載入符號表、常式、第 1 次處理的程式碼、再加上覆蓋所使用到的怪驅動程式 (10KB)，這樣一共是 140KB，第 1 次處理得以順利進行，執行完第 1 次的程式碼後，跳到重疊驅動程式，把第 2 次處理程式碼覆蓋在第 1 次處理程式碼的區塊上，然後控制權交給第 2 次處理程式碼，這樣一來，第 2 次的程式大小也只有 130KB，因此我們就可以順利完成兩次處理的工作 (圖 4-21)。

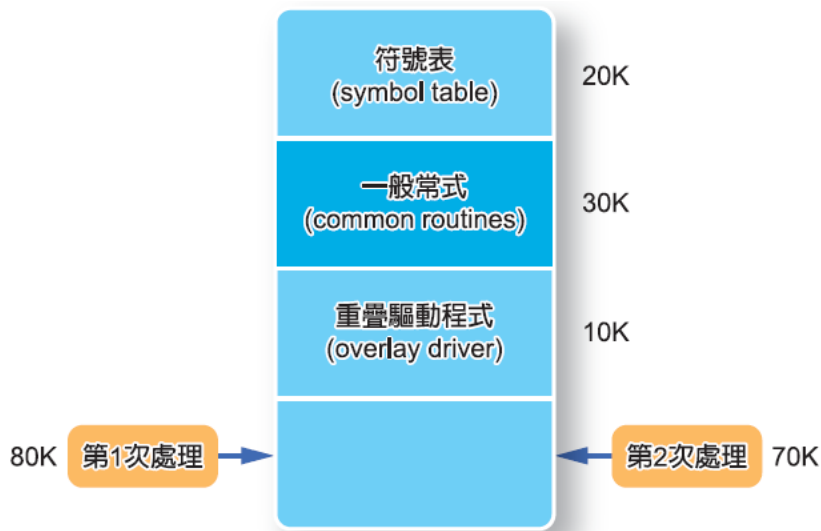


圖 4-21 利用覆蓋的方式使大小大於記憶體的程序得以執行

置換

由於程序可能在記憶體中被執行，然後被替換出來，因此程序可能會在記憶體與磁碟之間置換 (swapping) 備份儲存。舉個例子來說，在多元程式規劃系統中，如果我們使用依序循環排班方式來安排 CPU 的工作，則當時間間隔到時，記憶體管理程式便把剛剛結束的行程置換出去，然後換入別的行程 (圖 4-22)。

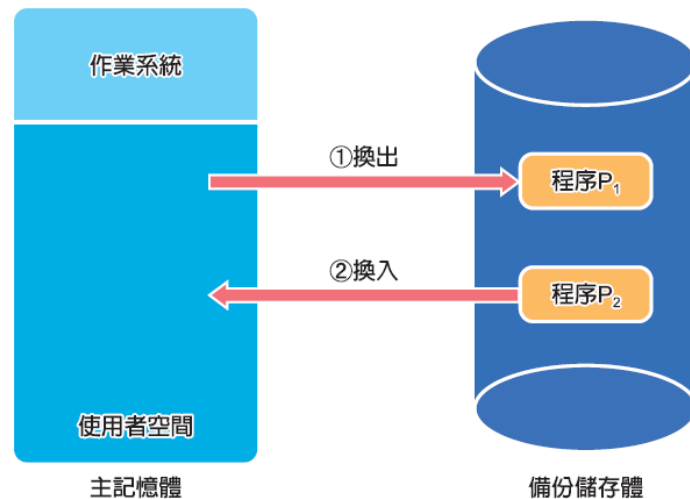


圖 4-22 利用磁碟當作備份的儲存體，用以置換兩個程序

這種方式還能搭配優先順序排班來使用，譬如說：當有較高優先順序的程序來到時，低優先順序的程序就先置換到磁碟去，等到高優先順序結束其執行時，低優先順序的程序再被置換進來。由於程序在記憶體上進進出出，因此有時置換也被稱作轉進轉出 (roll-in-roll-out) 方式。原則上，任一時間點應該都要有一程序在記憶體中被執行，(圖 4-23) 是置換方式中使用洋蔥皮演算法的例子：

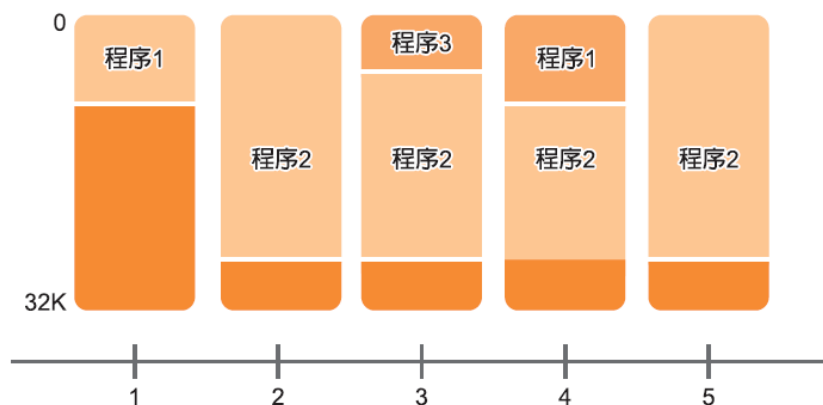


圖 4-23 洋蔥皮演算法示意圖

在第一時間，程序 1 配放入記憶體中，之後程序 2 進來，程序 1 備置換出記憶體，緊接著程序 3 進來，只需把程序 2 的一部份置換出去，之後程序 1 回來，把程序 3 的一部份換出，最後程序 2 回來繼續執行。

分頁

記憶體分頁 (memory paging) 是當前記憶體管理的重要機制，它將一個程序的記憶體需求分割成大小相同的頁面 (page)，以便更有效管理。另外，記憶體元件仍在變動中，**快閃記憶體** (flash memory) 在某些特定裝置有潛力取代硬碟和主記憶體。

4-5

檔案系統

對於使用者而言，最常接觸到作業系統的部分，就是檔案系統了。所謂檔案系統，是只負責存取和管理檔案資料的軟體。

對於使用者而言，檔案系統應該要具有操作簡單、安全可靠、能夠共享或是保密等特性；對於作業系統而言，則要能管好檔案使用空間，使存取更有效率等。檔案系統中，誠如我們所見，最重要的兩個部分就是檔案與目錄結構。

檔案中的資料是由建檔者去定義，可能是純文字檔案，或者是放原始程式碼，也可能是放報表之類的數字資料。檔案有幾個重要的屬性：

- **名稱**：符號式的檔名是給人辨識不同的檔案。
- **識別符號 (identifier)**：獨一無二的標籤，用來標示檔案系統內的檔案，通常是以數字來表示，是給作業存取所用，並非給使用者辨識。
- **型態**：提供使用者有關這個檔案的類型，譬如說是一份 Word 文件，或是一份 C++ 原始程式碼。
- **位置**：標示出這個檔案所在的磁碟裝置及目錄位置。
- **大小**：顯示該檔案目前的大小。
- **時間日期**：顯示檔案建立日期、修改日期、最後開啟日期等資訊。

所有的檔案屬性等資訊都存在目錄結構中。目錄結構包含檔名和識別符號，識別符號再指到其他檔案屬性。一個檔案的屬性大概需要 1KB 來記錄。接下來我們看看檔案的基本操作：

- **建立檔案**：建立檔案這工作可分成兩個步驟，首先，必須在磁碟中找到空間存放這個檔案；接著，必須在目錄中增添一個新的檔案項目。
- **寫入檔案**：要寫檔時，必須作系統呼叫，給予參數設定，如欲寫入的檔案及寫入的資訊。在目錄中會儲存指標指到檔案結尾，寫檔的時候就依照指標目前的位置把新資料家在後頭，最後儲存最新的指標位置。
- **讀取檔案**：讀檔時，同樣需要系統呼叫，指定要讀的檔案及其所在的位置。目錄裡也存放指標指向下次將被讀的位置，如果有資料又被讀出，指標的位置就在進行更新。
- **刪除檔案**：要刪除檔案時，我們必須搜尋目錄以找到檔案所在的位置，然後將檔案所佔用的空間釋放出來，並將登記在目錄裡的檔案項目刪除掉。

除了檔案之外，我們還要理解與檔案息息相關的目錄結構。目錄結構必須支援以下的操作功能：

- **搜尋**：搜尋是目錄結構中很重要的操作功能，我們必須要能夠搜尋一個目錄以找出某個檔案的所在位置。由於檔案有不同的屬性，因此我們也可以根據某一屬性特色進行檔案的搜尋。
- **建檔**：當有新檔案被建立時，目錄必須要能夠為其增加一個檔案項目。
- **刪除檔案**：在檔案被刪除時，必須要把該檔案相關的資訊刪除。
- **更改檔名**：當檔案名稱被使用者改變時，目錄結構裡所記載的檔案資訊也必須一起改變。

目錄結構中，最簡單的就是單層目錄（圖 4-24），所有的檔案都裝在同一層目錄中，一目瞭然。在單層目錄的情況中，所有的使用者檔案都放在一起，一旦檔案數目變多了，就開始有困擾。舉例來說，假設我們要交修課的第一個作業報告，可能大多數的人都會取名為 hw1.doc，然後放在 homework 資料夾。但是同一個目錄裡並不能有兩個相同的檔案名稱，於是就開始改檔名使得檔名衝突的情況不會發生。如果人數越來越多，取名的難度就越來越高，因此單層目錄雖然簡單卻不夠實用。

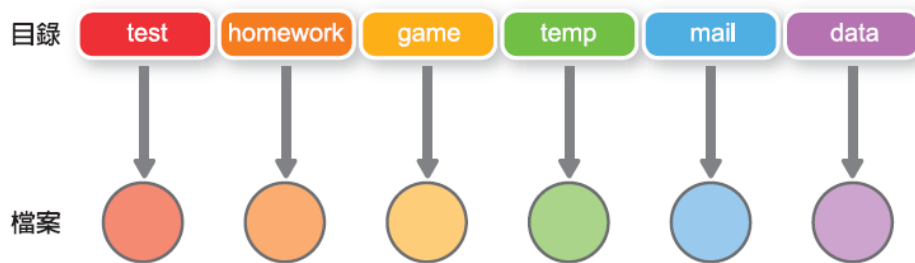


圖 4-24 單層目錄

使用雙層目錄（圖 4-25），可以改善上述的情況。最簡單的作法就是先為每一個使用者建立一個目錄夾，每個使用者的目錄夾裡的結構類似，但是只會顯示自己的檔案。當一個使用者想要開啟某個檔案的時候，只會搜尋他自己所屬的目錄夾裡，如此一來，不同的使用者就可以取相同的檔名而不會互相影響。

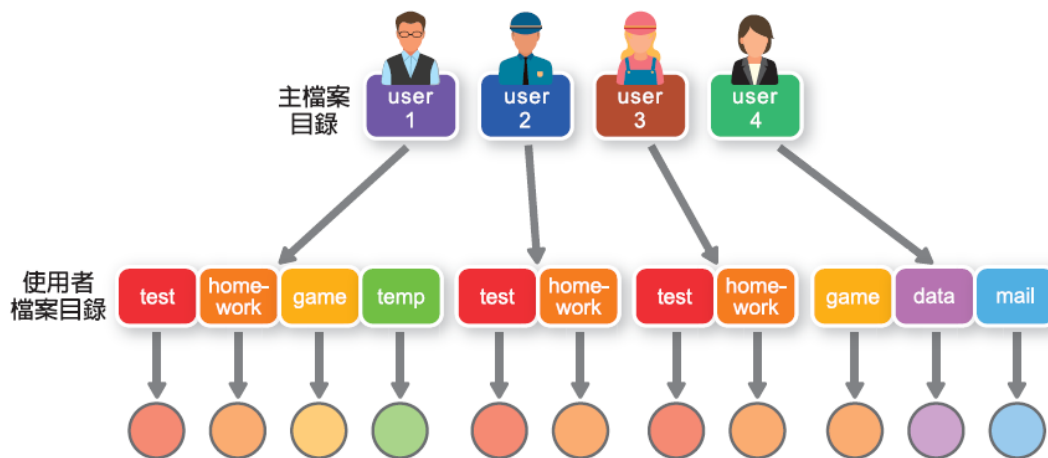


圖 4-25 雙層目錄

最後我們要來談絕對路徑和相對路徑。當使用者要開啟某個檔案的時候，如果給予系統的參數裡只有檔名，那麼系統會在**當前目錄**（current directory）中搜尋該檔案，找到就開啟，不然就給予錯誤訊息回應。可是如果使用者已知所要開啟的檔案不再當前目錄之下，那麼就必須給予該檔案的目錄資訊，也就是給予系統找到該檔案的路徑名稱。

路徑名稱可以分為絕對路徑和相對路徑。所謂絕對路徑是指由根部開始，一路指定目錄夾直到該檔案所在的目錄；相對路徑則是由當前目錄去定義要開啟的檔案所在的位置。舉個例子來看，假設我們有（圖 4-26）的目錄結構，並且當前目錄為 user1，則絕對路徑為 /user1/Homework/hw1.doc，相對路徑則為 Homework/hw1.doc（Windows 系列用的是倒斜線，如：Homework\hw1.doc）。

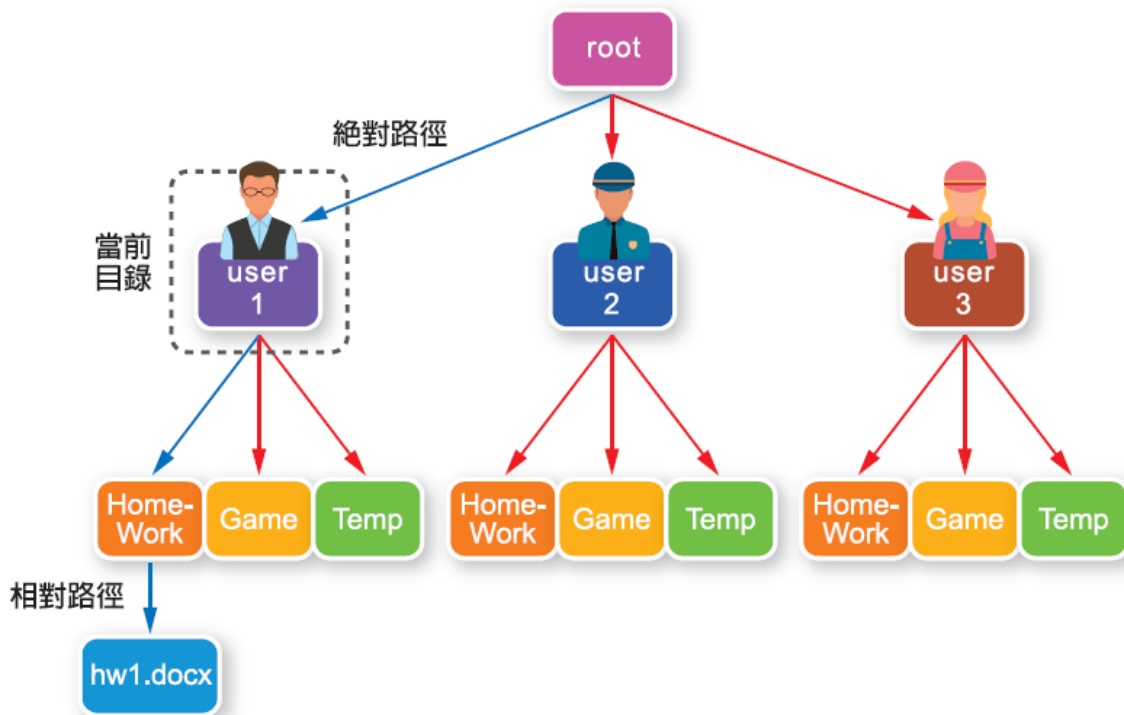


圖 4-26 絕對路徑與相對路徑

雖然目前看來，作業系統可說是微軟 Windows 的天下，可是隨著微軟的作風越來越強硬，有越來越多的國家鼓吹大家改使用開放原始碼的作業系統，此外，還有另外一批堅持用最漂亮的 Apple 電腦愛用族，使得 Mac OS 的市場佔有率也不可小觀。就讓我們來簡介這些熱門的作業系統。

Linux

個人電腦發展以後，微軟始終佔著霸業，如果說誰曾經讓比爾蓋茲心驚驚半夜睡不著，恐怕就只有 Linux 的創始人托瓦茲 (Linus Torvalds) 一人吧。在版權 (copyright) 高度受到宣揚的世代，托瓦茲高喊著反版權 (copyleft)，認為軟體應該透過分享的概念，才能更快速的傳遍世界，讓更多人受惠。

1991 年，當時托瓦茲還在芬蘭大學唸書，他把發展尚未成熟的作業系統 Linux 0.02 版本 (圖 4-27) 放上學校的 FTP 網站，並且到新聞群組去公告散佈這個消息。當時的一位大學生，就這樣造就了一番創舉。由於托瓦茲當時興起的念頭是想在 386 個人電腦上執行類似 Unix 的分時多工作業系統，加上當時現有的作業系統都不夠好用，於是他自己決定寫一個，結果受到廣大迴響，很多電腦高手都有同樣的需求。

托瓦茲所採用的策略是史托曼 (Richard Stallman) 所提倡的通用公共授權 (General Public License; GRL)，讓每個人都可以在 GRL 的授權之下，免費使用 Linux 程式碼，或者修改。由於 Linux 的開放性，加上一大批知名的、不知名的電腦駭客、編程人員加入到開發過程中來，Linux 逐漸成長起來。

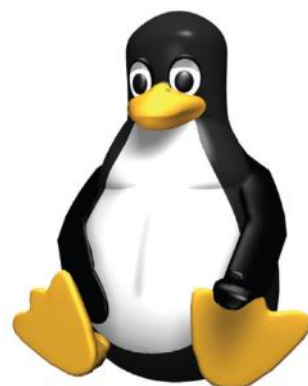


圖 4-27 Linux 的象徵圖案

雖然當初 Linux 0.02 版本的功能有限，並且漏洞百出，但是托瓦茲還是勇敢地把它放上 FTP 網站，帶著拋磚引玉的想法，讓大家一起把 Linux 變的更好。雖然有人擔心如果把原始碼公開出來，不就把作業系統的漏洞曝露出來了嗎？但是開放原始碼的社群則認為如果有漏洞，也可以透過在網路上曝露而讓高手很快地做出補救修正。就算有駭客故意放上植有後門的程式碼，也很快地就會被其他高手揪出。透過一群熱血的高手，整個 Linux 就能保持不斷地精進與最高安全性。

就技術上而言，Linux 有以下特點：

- 真正意義上的多工、多用戶作業系統。
- 支援數十種檔案系統格式。
- 提供了先進的網路支援。
- 採用先進的記憶體管理機制，更加有效地利用實體記憶體。
- 開放原始碼，用戶可以自己對系統進行改進。

如果你不想花錢買作業系統，害怕遇到 Windows 當機的藍色畫面，喜愛鑽研接觸作業系統的設定，Linux 應該會是一個很好的選擇。

Mac

電影「阿甘正傳」中有一個很可愛的畫面：阿甘對著旁邊聽他說故事的人述說著他的朋友幫他投資蘋果園，沒想到這年頭蘋果園竟然如此賺錢。結過鏡頭帶到信箋上，看到的是蘋果電腦的標誌（圖 4-28）。觀眾不禁莞爾：投資蘋果電腦當然賺錢呀。

很多時候在影片中出現的桌上型電腦。幾乎都是蘋果電腦。理由之一除了廠商贊助之外，蘋果電腦的優美外型恐怕也是無法抵擋（圖 4-29）。除了外型優美之外，Mac OS 使用者介面（圖 4-30）也是美得令人驚豔。



圖 4-28 蘋果電腦的標誌



圖 4-29 蘋果電腦的造型



圖 4-30 Mac OS

Mac OS 除了穩定之外，最令人著稱的就是它的使用者介面。不論是按鈕，顯示圖示，或者是邊框的設計，都是經過精心設計，讓使用者擁有最賞心悅目的畫面。另外，Mac OS 開發團隊花許多人力資源再研究使用者介面的設計。所謂使用者介面的設計是指研究人類的行為與操作電腦之間的關係，讓使用者能夠盡可能方便地找到如何使用某功能，或者讓使用者很快捷地選取常用的軟體。此外，很多美術設計的部門都會使用 Mac OS，因其有較佳的設計軟體可供使用。

雖然 Mac OS 的畫面很漂亮，性能也很棒，然而這個世界上有很多是離不開 Windows 的人，卻又不甘心使用者介面不敵 Mac OS，因此很多人會上往下載**佈景主題** (theme)，在 Windows 作業系統中，將使用者介面改的像 Mac OS 一樣漂亮，姑且就稱之為「偽 Mac」吧！

Windows

談到 Windows，實在是不得不用霸業來形容。雖然很多人反微軟，反比爾蓋茲，然而同時，卻又不得不佩服比爾蓋茲獨到的眼光，咒罵比爾蓋茲如惡魔壟斷市場時，卻又不得不讚美他投入大量金錢於慈善醫療事業。整個 Windows 是個龐大的家族。

很多人誤以為 Windows 源自於 90 年代，其實 1985 年就推出了 Windows 1.0 版本。在 Windows 之前，使用者用的是 MS-DOS 命令列模式，只能靠著在命令列中以文字模式操作電腦。有了 Windows 之後，使用者開始能用滑鼠來點選執行工作。此外，Windows 1.0 中就已經提供了目前還看得到的桌面應用程式，如時間、月曆、筆記本、小算盤等功能。

微軟自 1990 年推出 Windows 3.0 後，除了新硬體的支援、效能上的提升，還提供了如程式管理員、檔案管理員、印表機管理等功能。另外**軟體開發工具** (Software Development Kit ; SDK) 的發行，也使得軟體工程師可以開發 Windows 作業系統上的各式應用。這使得 Windows 系統的普及率愈來愈高。而在使用者介面設計方面，從 Windows 95 出現所謂的「開始功能表」後，就一直延用至今。之後每間隔三至五年，微軟就會推出新版的 Windows 系統，以提升系統效能、強化系統安全、或是開發新的功能。時至今已推出至 Windows 11。近期 Windows 系統的畫面截圖 (圖 4-31) 所示。

作業系統的競爭愈來愈激烈。隨著開放原始碼社群的活躍如 Linux 作業系統的普及，微軟在 Windows 10 系統中甚至推出了 WSL (Windows Subsystem for Linux) 功能。簡而言之，就是在 Windows 作業系統中內建 Linux 作業系統的**核心** (kernel)，使得原本只能在 Linux 系統中運作的程式，也可以不需要修改就可以直接搬到 Windows 上，透過 WSL 的機制運行。此外，Windows 的開發環境也對開放原始碼社群愈來愈友善。除了官方 github (<https://github.com/microsoft>) 提供許多相關軟體的開源資訊，微軟提供的免費 Visual Studio Code 程式碼整合開發介面也頗受好評。種種改變也使得 Windows 系統在開源社群和伺服器市場更具有競爭力。



Windows 7 的使用者介面



Windows 8 的使用者介面



Windows 10 的使用者介面



Windows 11 的使用者介面

圖 4-31 近期 Windows 系統的使用者介面

2021 年 10 月，Windows 推出 11 版作業系統。除了重新設計大幅修改的介面、強化觸控介面和行動設備的支援外，筆者認為三個令人玩味的有趣特色是 (1) Windows 終於徹底拋棄 Internet Explorer，僅保留與 Chrome 瀏覽器相同核心 (Chromium) 的 Edge 瀏覽器。(2) TPM 2.0 安全模組的導入使用，透過硬體安全機制，讓系統整體可以更為安全。以及 (3) 實作更完整的適用於 Linux 的 Windows 子系統 (Windows Subsystem for Linux ; WSL)，內建可支援圖形介面的 Linux 核心和工具。

微軟 Windows 作業系統上最受歡迎且最不容易被取代的軟體大概就是他們自家的 Office 軟體。Office 軟體的成功也使得微軟 Windows 作業系統的普及率難以撼動。雖然開源社群或是其他的商業公司也有推出類似的軟體，如 OpenOffice、LibreOffice、EasyOffice 甚至 Apple 推出的 Pages 和 Keynote，都還沒辦法完全取代微軟的 Office 軟體。不過微軟的 Office 軟體還是以使用自訂的封閉格式為主。在開源文化和開放文件格式 (Open Document Format ; ODF) 興起，以及其他作業系統使用者大幅成長的情況下，未來軟體要維持 Office 軟體和 Windows 作業系統霸主的地位可能還是得小心經營。

4-6

行動裝置作業系統

行動裝置從早期的傳統手機演進到现在的智慧型手機和平板，其功能也愈來愈強大複雜。因此，現在的行動裝置上，大多也有搭配適合的作業系統。在這一節裡，我們將針對幾個常見的行動裝置作業系統作簡單的介紹，包括 Android、iOS、以及 ChromeOS。

Android

在不久的幾年前，人們身上會隨身攜帶的手持式裝置大概只有手機，而它的功能只能用來打電話和送簡訊。隨著科技的演進，手持式裝置的硬體規格和功能愈來愈強大，現代人可以不用再坐在桌子前面操作電腦，取而代之的是透過各式各樣手持行動裝置，在任何時間、任何地點，進行原本只能在個人電腦上進行的工作、教育以及娛樂等活動。而為了讓這些強大的手持式裝置得以運作順暢，這些裝置上面也必須要有足以支持複雜系統運作的作業系統。而 Android 就是一個專門為嵌入式和手持式裝置開放的作業系統之一 (圖 4-32)。

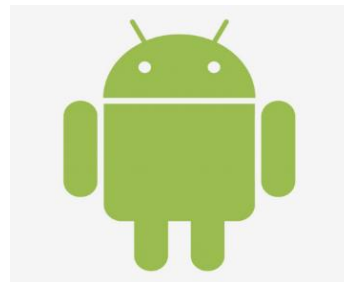


圖 4-32 Android 的象徵圖案

Android 一開始是由一家叫做 Android 的公司自 2003 年起開始進行研發的作業系統。而這個作業系統是以像數位相機這樣的小型嵌入式系統為目標。2005 年，Google 買下了 Android 這一間公司，且仍然以 Android 為名稱，繼續進行相關的研發。2007 年 11 月 5 日，Google 對外正式宣佈將 Android 作業系統用於手機，同時也宣稱 Android 將會是一個免費且開放原始碼的作業系統。而第一個搭配 Android 作業系統 1.0 版的手機，也於 2008 年 9 月，以美國電信商 T-Mobile 之名推出型號為 G1 的手機。自 2008 年起，Android 已經發佈了許多版本，而令人覺得有趣的是，自 Android 1.5 版本起，所有主要發行的 Android 作業系統版本都有對應的甜點別名！Android 發行 10 年後，Google 終於再也想不出新的甜點名字了 (也可能是厭倦了，或是受到 iPhone X 的刺激)！所以當 Android 10 推出時，就非常直接了當地用了「Android 10」做為版本的名稱。(表 4-9) 列出的是 Android 的各個主要版本。而我們也可以看出，Android 其實是一個發展迅速，且改變劇烈的作業系統。隨著新版本的推出，開發工具的 API 等級也一直提升。也因此，Android 軟體開發人員，也需要小心挑選開發工具的 API 等級，以支援最多的使用者。

表 4-9 Android 版本列表

版本	發行時間	別名	API 等級
Android 1.5	2009 年 4 月	紙杯蛋糕 (cupcake)	3
Android 1.6	2009 年 9 月	甜甜圈 (donut)	4
Android 2.0	2009 年 10 月	閃電泡芙 (éclair)	5
Android 2.2	2010 年 6 月	霜凍優格 (froyo)	8
Android 2.3	2010 年 12 月	薑餅 (gingerbread)	9
Android 3.0	2011 年 2 月	蜂巢 (honeycomb)	11
Android 4.0	2011 年 5 月	冰淇淋三明治 (ice cream sandwich)	14
Android 4.1	2012 年 7 月	雷根糖 (jelly bean)	16
Android 4.4	2013 年 10 月	奇巧 (kit kat)	19
Android 5.0	2014 年 11 月	棒棒糖 (lollipop)	21
Android 6.0	2015 年 10 月	棉花糖 (Marshmallow)	23
Android 7.0	2016 年 8 月	牛軋糖 (Nougat)	24
Android 8.0	2017 年 8 月	奧利奧 (Oreo)	26
Android 9.0	2018 年 8 月	派 (Pie)	28
Android 10.0	2019 年 9 月	10	29
Android 11.0	2020 年 9 月	11	30
Android 12.0	2021 年 10 月	12	31
Android 13.0	2022 年 8 月	13	33
Android 14.0	2023 年 10 月	14	34
Android 15.0	2024 年 10 月	15	35

Android 也是一個基於 Linux 核心開發的作業系統，但它和一般的 GNU/Linux 作業系統的差異很大。除了核心和一些基本的工具外，Android 系統裡使用的視窗圖形介面系統以及系統函式庫，都是特製非標準的函式庫。這個函式庫與大部分 POSIX 函式相容，但並不是完全相同。這也使得某一些 GNU/Linux 環境下的工具不易被移植到 Android 系統裡。此外，Android 的應用程式大都是使用 Java 程式語言編寫，而執行時是透過一個名為 Dalvik 的類 Java 虛擬機器 (virtual machine) 來執行。

Android 有一套自己的系統資源權限管理機制，且所有的程式執行都是在獨立的沙盒 (sandbox) 內執行。自 Android 4.4 版起，Google 更提出了 Android ART (Android Runtime)，嘗試取代舊有效率較差的 Dalvik 執行環境，使用者可以自行選擇使用傳統的 Dalvik 或是新的 ART 執行環境。然而自 Android 5.0，ART 則成為內建且唯一的執行環境。雖然較之下，Android 是個年輕的系統，但和當初最原始的 Android 系統比較起來，今日的 Android 作業系統幾乎把所有重要系統元件都更新了一輪。

隨著版本的演進，Android 作業系統的功能愈來愈穩定完整，操作介面愈來愈美觀、且系統效率也愈來愈好。(圖 4-33) 展示的是幾個 Android 操作介面的畫面。而 Android 作業系統受歡迎的程度，若以目前的市佔率，據國際數據資訊 (IDC) 公司在 2024 年一月的預估，市場上售出的智慧型手機裡，有超過七成五的智慧型手機都是使用 Android 系統！Android 作業系統可以說是目前最多人使用的智慧型手機作業系統。有興趣的讀者也可以統計看看，到底周遭的朋友們都是使用什麼樣的智慧型手機作業系統呢？

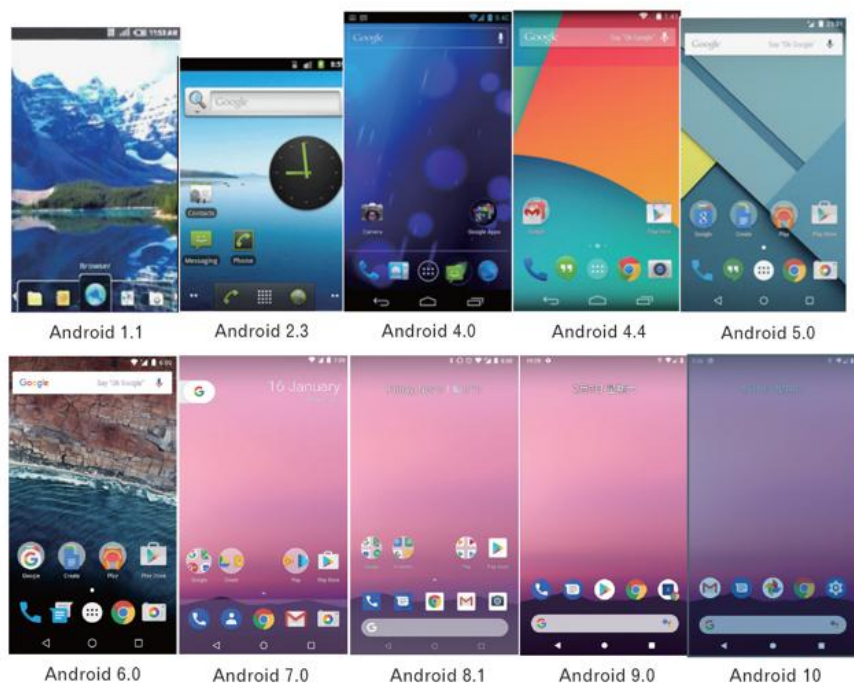


圖 4-33 Android 操作介面的畫面

iOS

有部份的讀者可能是忠實的蘋果迷，而除了蘋果的桌上型和筆記型電腦外，另一個大受歡迎的產品就是蘋果的智慧型手機 iPhone 了！一般的蘋果電腦使用的作業系統是 Mac OS X，而 iPhone 手機上所使用的作業系統則是 iOS。雖然 Mac OS X 和 iOS 看起來外觀和介面有所不同，但其實它們的作業系統核心是同一套：都是基於 Darwin 核心所建構起來的作業系統。

iOS 最早版本是在 2007 年發行的 iOS 1.0 版，它最早是用在第一代的 iPhone 手機和 iPod Touch 裝置上。而隨著新的蘋果手機的推出，蘋果電腦公司也不斷的更新他們的作業系統，目前已經到 17.x 系列的版本。就外觀上而言，iOS 作業系統一直以來都沒有太大的變化，(圖 4-34) 所示。

和蘋果電腦一樣，iOS 作業系統同樣也只能在蘋果自家的智慧型手機上才能使用。不論軟硬體如何升級，iOS 也盡量保留給使用者相同的風格和操作習慣！



圖 4-34 iOS 作業系統主業畫面截圖

使用者常常拿 Android 系統和蘋果公司的 iOS 系統來進行比較，這二個系統同樣都是給行動裝置如手機和平板使用的作業系統，然而這二個系統在理念上有很大的差異。相較之下，iOS 系統算是比較封閉的系統。雖然二者的官方網站都有提供「軟體市集」的服務（Google 的 Play 服務以及蘋果的 App Store 服務）供使用者下載和安裝各式各樣的應用程式。

iOS 是不允許使用者從第三方網站安裝未經蘋果公司檢驗的軟體。同時，蘋果公司對於其軟體市集上架的軟體檢驗時間較長、標準也較為嚴格。雖然系統較為封閉，一般也認為 iOS 的環境可能是比較安全的。

最後，我們以（表 4-10）比較二種最熱門的智慧型手機作業系統。當然青菜蘿蔔各有喜好，從使用者的角度來說，不論作業系統功能如何，還是自己用得開心最重要啦！

表 4-10 比較熱門的行動裝置作業系統：Google Android VS Apple

	Android	iOS
系統本質	開放、多元、有彈性	簡潔、易用、穩定
系統核心	Linux	OS X (Darwin)
軟體開發語言	C/C++/Java/Kotlin	C/C++/Objective-C/Swift
系統客製化	容易	非常有限。除非越獄
可使用記憶卡	依裝置廠牌型號而有所不同	無法使用

ChromeOS

Chrome OS 是由 Google 針對上網裝置推出的輕型作業系統。Chrome OS 的系統核心是基於 Linux 核心發展的。而上層的應用程式都是以網頁應用程式的概念呈現，也就是可以透過 Google 的 Chrome 瀏覽器執行。Chrome OS 的理念就是：使用者不需要安裝任何應用程式，只要有網頁瀏覽器，所有的應用程式都可以在雲端裡執行。也因此，Chrome OS 其實沒辦法安裝一般的應用程式。而雲端上的應用程式（如 Google 的各式服務）在 Chrome OS 裡執行時，也包裝得好像就是本機的應用程式一樣。此外，使用者必須要有 Google 帳號，才得以登入和使用 Chrome OS。Chrome OS 的介面（圖 4-35）所示。

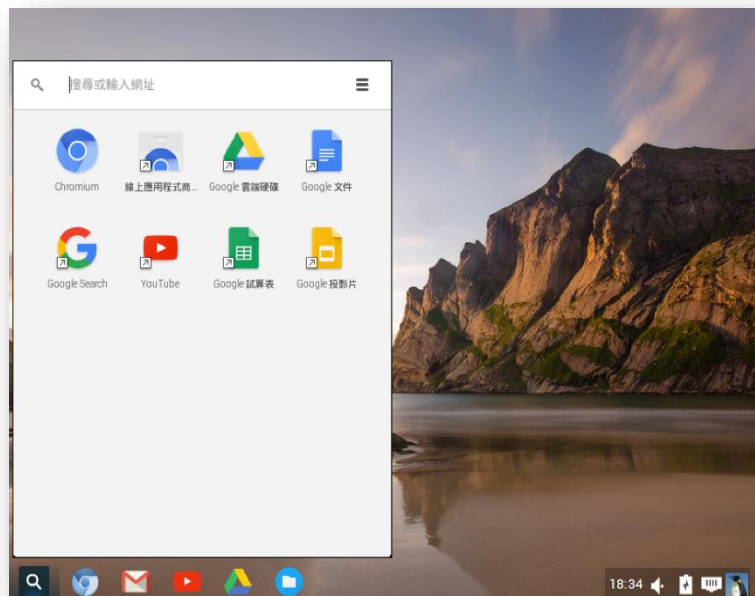


圖 4-35 Chrome OS 的桌面截圖

Chrome OS 最早的版本是在 2009 年 7 月發行。然而，使用者並沒有辦法自行下載安裝。Chrome OS 多半搭配特定的硬體發行，市面上販售的 Chrome Book 就是搭載 Chrome OS 的筆記型電腦。另外，Google 自行推出的 Chromecast 也是使用精簡版的 Chrome OS 系統。2009 年 11 月，Google 把 Chrome OS 的原始碼釋出，也就是開放原始碼 Chromium OS 計畫的核心。由於是開放原始碼的計畫，任何人都可以把 Chromium OS 的程式碼下載回來，自行修改、編譯、執行和發佈。不過 Chromium OS 計畫本身主要是鎖定給開發人員使用。計畫本身並沒有提供安裝硬碟。不過，網路上有許多熱心人士或是公司發行自製的 Chromium OS 安裝光碟映像檔。其中，發行 Ubuntu Linux 的 Canonical 公司就曾推出整合 Chrome OS 介面的 Ubuntu Linux 版本。這個名為 Chromixium OS 的計畫就有提供可安裝的光碟映像檔。不過由於部分軟體檔案授權的關係，目前這些基於 Chromium OS 的免費系統已經沒有在維護和提供下載。